Here is the first part of the project description

Mini-project 2: 1D reaction-transport model

Develop a model for oxygen transport and organic matter (OM) consumption by bacteria in an open-top wastewater "polishing" lagoon without inflow or outflow. Assume that the wastewater is stagnant (so transport is by diffusion only) and contains OM that is consumed by aerobic bacteria only. In the model you need to include at least dissolved oxygen and OM as state variables and you can assume that OM degradation is a second-order reaction. You can also assume that it takes 1 gram of oxygen to degrade 1 gram of OM. (In fact this is usually done in the wastewater treatment field because organic matter is expressed in "oxygen demand" units.)

Tasks

1. Model development

The first task is to develop the 1D model of the wastewater tank with oxygen and OM as state variables. Include a sketch of the model and the system boundary. Describe how you formulated the model and what assumptions you made, including information on the coordinate system, simplification of the governing equations, boundary conditions, and initial conditions. For your Python code, you should use a module-based approach for your model (define a model function in a `*.py` module and then write the code for verification, validation, and application in separate `*.py` scripts). Be sure to comment your code enough that someone could understand it without input from you! That information should be present in docstrings and single line comments.

And model formulation steps presented near the start of the course

```
# Steps in model formulation

1. Initial concepts and assumptions (boundary, state variables, sketches)
2. Balance/conservation equations
3. Constitutive equations (transfer or rate equations)
4. Linking equations
5. Spatial or other simplifications
6. Initial or boundary conditions
7. Governing equation
8. Analytical or numerical solution
```

And later on for PDE problems

Algorithms for solving text problems

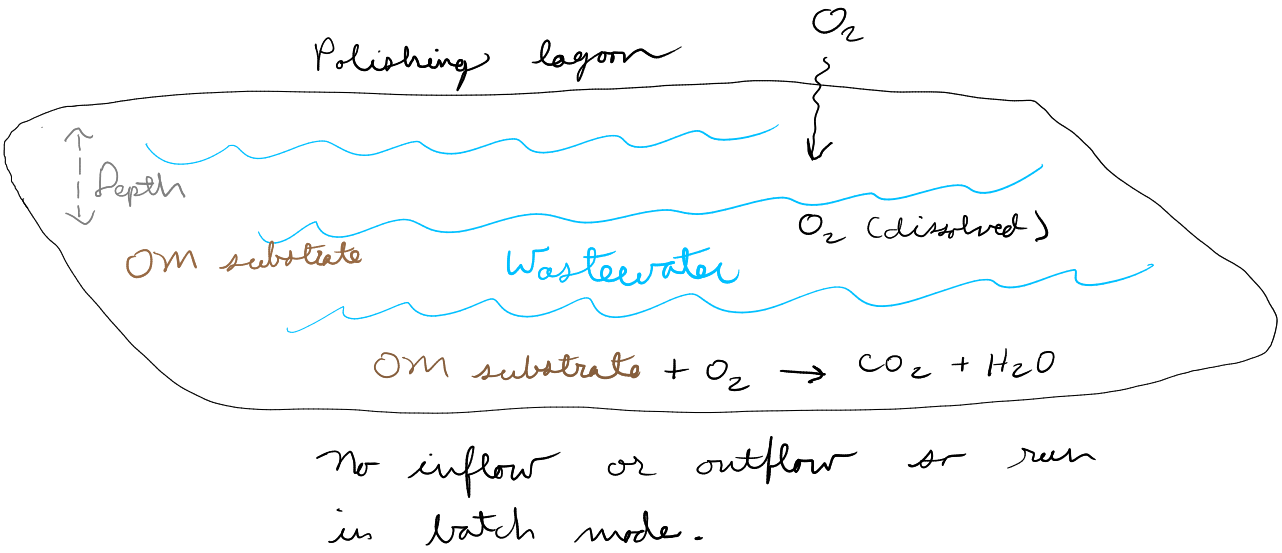
1. Select PDE coordinate system
2. Eliminate uneccesary terms to get the governing equation
3. Reformulate to discretized form (for method of lines)
4. Figure out what the BC's and IC's are
5. Implement in python by handling interior and BC nodes separately

Here we will do a combination. The list below starts with 0 so the numbers match those used for the PDE approach above.

0. Initial concepts, assumptions, and simplifications (boundaries, state variables, sketches).
1. Select PDE coordinate system and appropriate governing equation (GE).
2. Eliminate unnecessary terms from GE.
3. Reformulate to discretized form.
4. Sort out boundary conditions and initial conditions.
5. Implement in Python, with different code for interior and boundary nodes.

0. Initial concepts and assumptions, including a sketch and the conceptual model

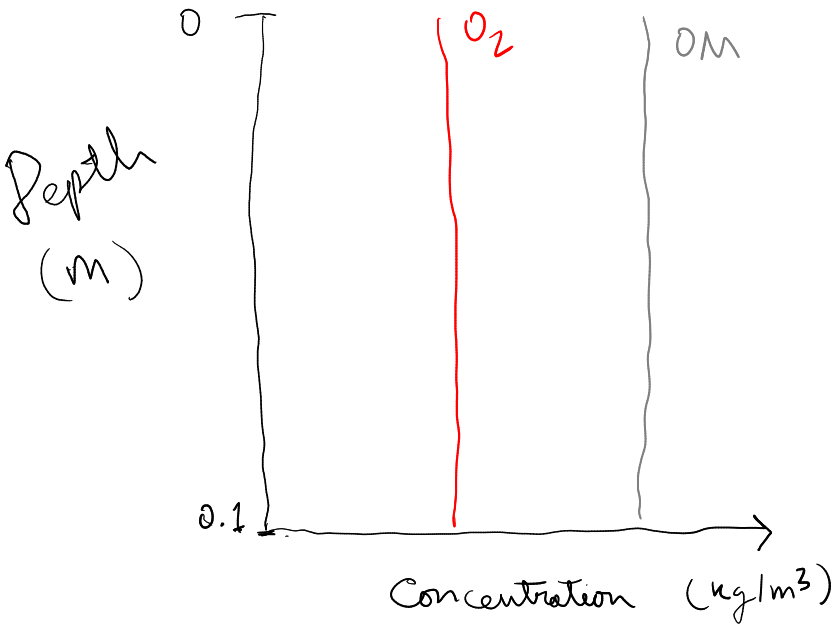
Sketch and conceptual model



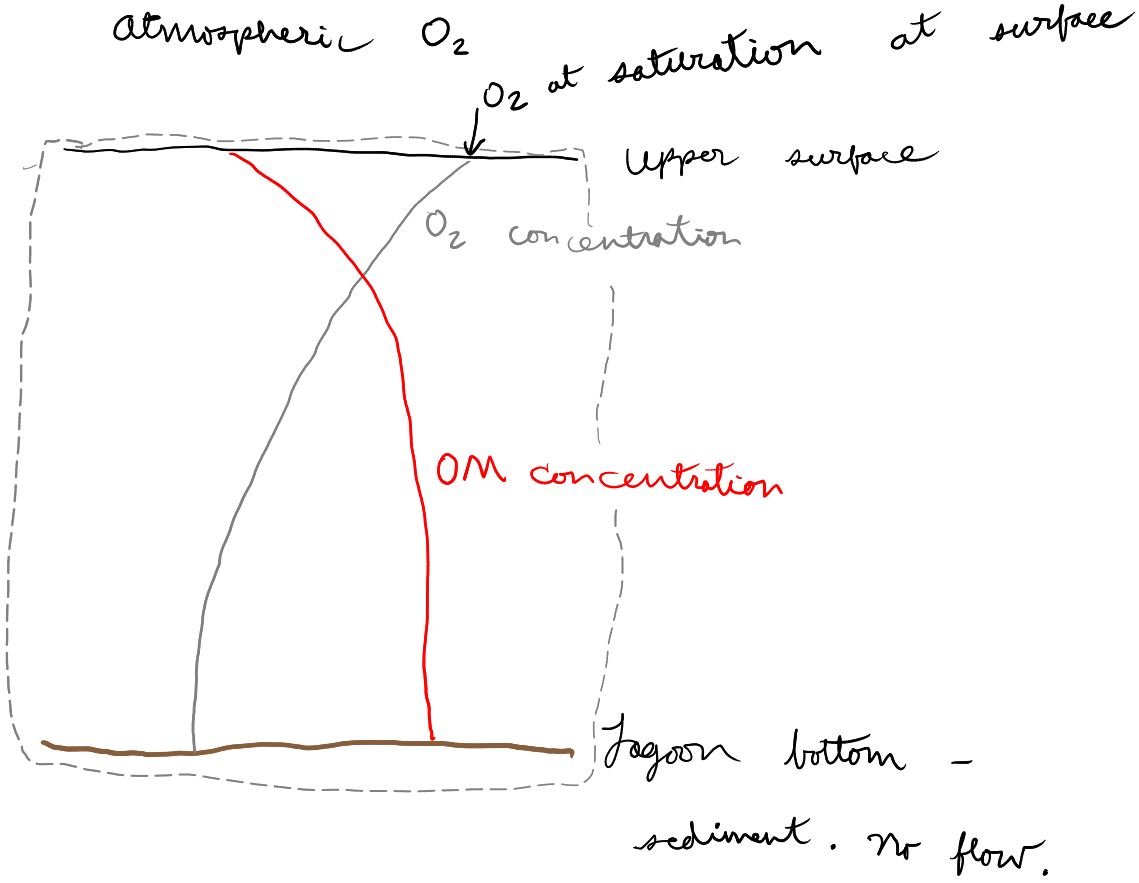
State variables are concentrations of dissolved O2 and substrate OM.

We'll include some simplifications here. Some are given in the project description but let's justify them anyway.

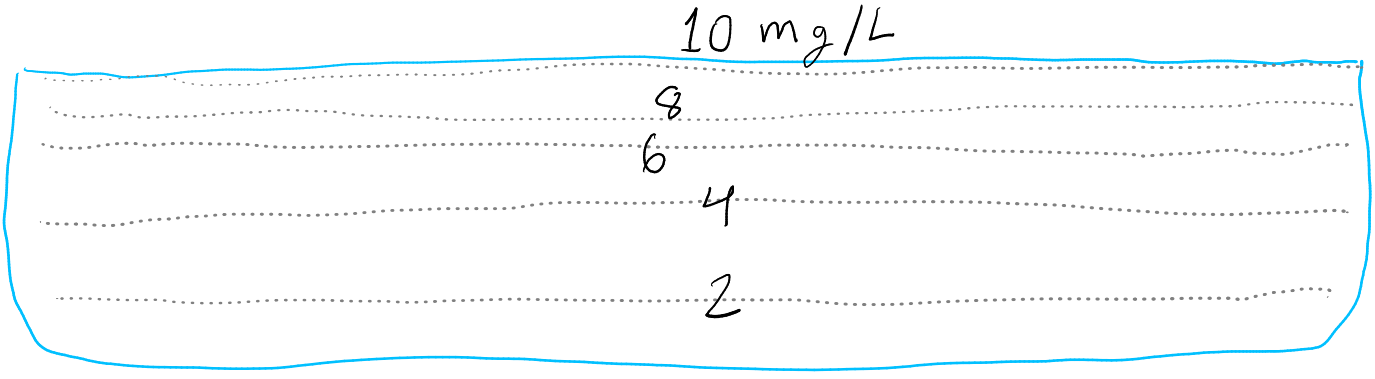
Right from the start let's plan on developing a 1D model. This is a reasonable simplification. Horizontal gradients in OM or O2 should not exist if the system is run in a batch mode and well-mixed at the start. So our single dimension is depth.



Of course we do not expect straight lines for concentration (uniform concentration). And we should not expect constant lines either (steady-state) because the OM will be depleted over time, changing OM concentration, and therefore O2 consumption rate, and therefore O2 diffusion. Here are some more details.



Our model boundary is shown by the dashed gray line. We can simulate a column of water without thinking about how wide it is (or its volume) because this is a 1D model. So we assume that state variable values depend only on depth and time--they are always uniform over horizontal space. So lines of constant O2 concentration might look like this over the lagoon's width at some particular time, for example. We don't have to think about horizontal space in this 1D model.



Because transport is thought to be by diffusion, and
  * diffusion in liquid is much slower than in gas, and
  * O2 is not very soluble in water,
we can assume all mass transfer resistance is in the water. So we can use the saturation O2 concentration at the top as a model boundary condition.

1. Find the governing equation.
Here is the appropriate governing equation from the course text book:

A.1 Rectangular coordinates

Mass balance for component A (ρ and D_{AB} constant)

$$\frac{\partial C_A}{\partial t} + v_x\frac{\partial C_A}{\partial x} + v_y\frac{\partial C_A}{\partial y} + v_z\frac{\partial C_A}{\partial z} = D_{AB}\left(\frac{\partial^2 C_A}{\partial x^2} + \frac{\partial^2 C_A}{\partial y^2} + \frac{\partial^2 C_A}{\partial z^2}\right) + R_A.$$

2. Eliminate terms.
For a 1D model we can eliminate all y and z terms if we call our depth dimension "x". (It is OK to call it y or z too.) See the terms crossed out in red. And we can eliminate add advection terms because we don't have advection in this model. See terms crossed out in green.

A.1 Rectangular coordinates

Mass balance for component A (ρ and D_{AB} constant)

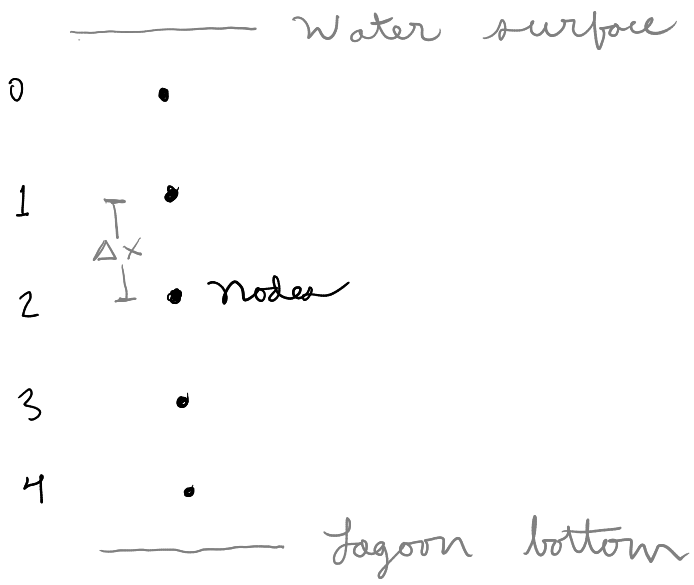
$$\frac{\partial C_A}{\partial t} + v_x\frac{\partial C_A}{\partial x}^{0} + v_y\frac{\partial C_A}{\partial y}^{0} + v_z\frac{\partial C_A}{\partial z}^{0} = D_{AB}\left(\frac{\partial^2 C_A}{\partial x^2} + \frac{\partial^2 C_A}{\partial y^2}^{0} + \frac{\partial^2 C_A}{\partial z^2}^{0}\right) + R_A.$$

That gives up this, much simpler GE:

$$\frac{\partial C_{O_2}}{\partial t} = D \cdot \frac{\partial^2 C_{O_2}}{\partial x^2} + r$$

3. Discretize

So let's conceptually divide the domain into nodes and apply the central difference formula. In the diagram let's use a small number of nodes--five.



Then discretize and use the central difference formula. Remember that for each cell, the time derivative of concentration should have a diffusion and a reaction component.

$$\frac{dc[i]}{dt} = D \cdot (c[i+1] - 2 \cdot c[i] - c[i-1]) / \Delta x^2 - r$$

This equation applies to all nodes, at least all interior nodes, for both oxygen and OM, as long as we recognize that "c" means concentration of oxygen in one case and OM in another.

We need an expression for the reaction rate, which we can get from the project description, where it is called a second-order reaction. So:

$$r = k \cdot c_{O_2} \cdot c_{OM}$$

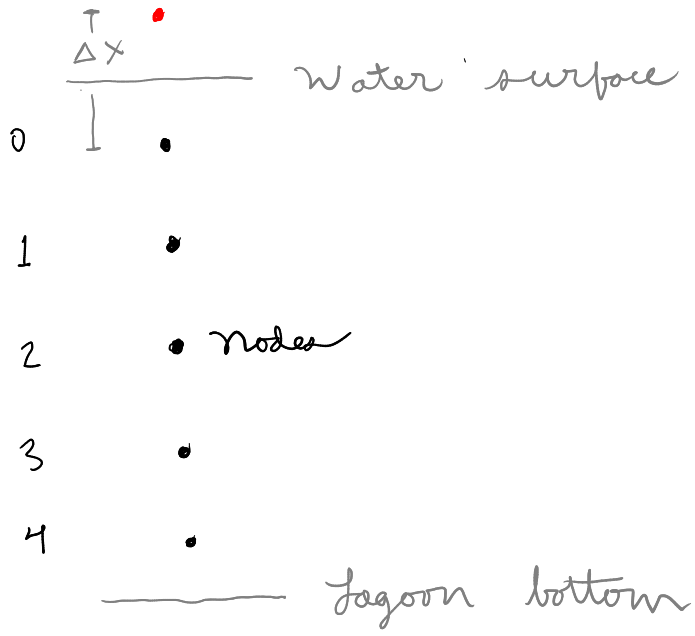
Putting all this together gives these two equations for interior cells for oxygen and OM:

$$\frac{dC_{O_2}[i]}{dt} = D\left(C_{O_2}[i+1] - 2 \cdot C_{O_2}[i] + C_{O_2}[i-1]\right)/\Delta x^2 - k \cdot C_{O_2}[i] \cdot C_{OM}[i]$$

That is kind of a mix of mathematical symbols and code, or "pseudocode". In Python it would be something like this: dcdt[i] = D * (c[i+1] - 2 * c[i] + c[i-1])/dx**2.

4. Boundary and initial conditions

How about the boundaries? We have two different types of boundary conditions. For O2, we have a fixed atmospheric concentration (or the equivalent saturation concentration) at the upper surface, with transport into the water. This is a Dirichlet BC and we can program it using a "ghost point". We can add this ghost point to the diagram in red:



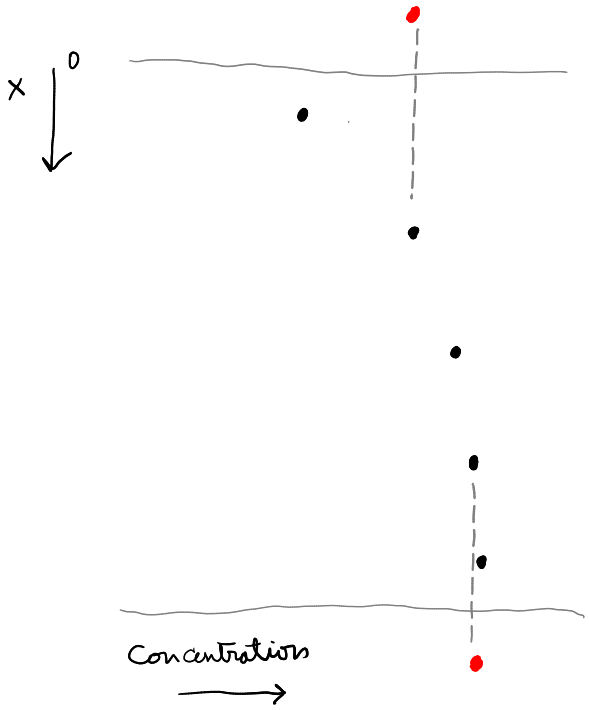
Now, it is more accurate to take that distance as half of the node spacing, $\frac{\Delta x}{2}$

but this increases the code complexity and has a negligible effect once our grid is fine enough. And it is not typically done with this node-based approach. In the code you can see we used dx, i.e., a whole distance not half.

For OM, and for O2 at the bottom, we have no flux, or a Neumann BC where the space derivative is zero. So for all three of these, we create a "ghost point" with a value equal to the concentration of the node 1 in from the boundary. That shuts off flux because the concentration difference (and therefore gradient) is zero. (You might wonder why we don't use the nodes at the boundary! Good question, and you actually can do this, but the approach we used is supposed to be more accurate, depending on who you believe.)

Note that we only need these Neumann BC ghost points if we are going to use the same code for interior and boundary nodes. Otherwise we could simplify the resulting equations. You can see in the code we kept the same central difference form but have the calculations for boundary and interior nodes on different lines.

We can visualize all this with our node diagram but with horizontal position indicating concentration, and putting ghost points in red.



Check out the solution modules to see how all this was actually implemented in Python.

How about the initial conditions?

Initial conditions (that is, values of our state variables at the start) could vary. For the model validation part we do have some info:

3. Model validation

An engineer at the wastewater plant measured the concentration of dissolved oxygen near the bottom of a 10 cm deep lagoon over a couple months, starting right after filling. The measurement data are in the `meas_02.csv` file. The filling process effectively aerated the wastewater at the start, but as you can see in the measurements, dissolved oxygen concentration declined over the following days. The initial OM concentration was 0.02 kg/m3 (20 mg/L). Carry out graphical and quantitative model validation using these measurements.

So the initial O2 concentration should be at or near saturation. And initial OM 0.02 kg/m3.

5. Implementation

See the modules for how we did this. Recognize that there is a lot of variability in exactly how the same mathematical model could be implemented! For example, dcdt for boundary nodes could be part of the same loop used for the interior cells if the state variable array is appended/prepended with ghost point values. And slicing could be used instead of a for loop. The Python code shows only some of the possibilities. If you are unsure whether your solution is equivalent to ours, please ask!